

MDM-221-COM : Digital Electronics and Login Design

(2024 Pattern) (Semester - III)

Unit 1	Boolean Algebra and Simplification Techniques	Marks
Boolean Algebra: Basic theorems and properties of Boolean algebra, DeMorgan's rules. Signed Binary number representation: Signed Magnitude, 1's complement and 2's complement representation. Simplification Techniques: Sum of product and Product of sum form, Minimization of SOP and POS using K-map. Minimization of Boolean function using K-map (up to 4 variables).		14

Introduction and Definition

Boolean Algebra, named after its inventor George Boole, is a branch of algebra that deals with variables that can hold only two possible values: **true** or **false**. In the context of digital electronics and computer systems, these values are represented by the binary digits **1** (for true) and **0** (for false).

Boolean algebra is the fundamental mathematical system that underpins all digital logic. It provides a framework for analyzing and simplifying digital circuits. The primary operations in Boolean algebra are:

- **AND (Logical Conjunction):** Represented by a dot ($\cdot$) or by placing variables next to each other (e.g., $A \cdot B$ or AB). The output is 1 only if *all* inputs are 1.
- **OR (Logical Disjunction):** Represented by a plus sign ($+$). The output is 1 if *at least one* of the inputs is 1.
- **NOT (Logical Negation/Inversion):** Represented by an apostrophe ($'$), a bar over the variable ($\bar{A}$), or a tilde ($\sim$). It inverts the input value. The output is 1 if the input is 0, and 0 if the input is 1.

Fundamental Properties and Theorems

These theorems are rules that allow us to manipulate and simplify Boolean expressions, much like rules in ordinary algebra.

1. Commutative Law

This law states that the order of variables does not affect the outcome of an AND or OR operation.

- **Definition:**
 - For OR: $A + B = B + A$
 - For AND: $A \cdot B = B \cdot A$
- **Explanation:** Whether you are ORing A and B, or B and A, the result is the same. Similarly, ANDing A and B is identical to ANDing B and A. This is analogous to how $3 + 4 = 4 + 3$ in regular arithmetic.
- **Example:** If $A=1$, $B=0$:
 - $1 + 0 = 1$ and $0 + 1 = 1$. The results are identical.
 - $1 \cdot 0 = 0$ and $0 \cdot 1 = 0$. The results are identical.

2. Associative Law

This law states that the grouping of variables in an expression with the same operator does not affect the outcome.

- **Definition:**
 - For OR: $(A + B) + C = A + (B + C)$
 - For AND: $(A \cdot B) \cdot C = A \cdot (B \cdot C)$
- **Explanation:** This law allows us to remove parentheses when combining multiple variables with the same operator, making expressions like $A + B + C$ unambiguous.
- **Example:** If $A=1$, $B=0$, $C=1$:
 - $(1 + 0) + 1 = 1 + 1 = 1$
 - $1 + (0 + 1) = 1 + 1 = 1$

3. Distributive Law

This law allows for the factoring or expansion of expressions.

- **Definition:**
 - $A \cdot (B + C) = (A \cdot B) + (A \cdot C)$
 - $A + (B \cdot C) = (A + B) \cdot (A + C)$ (This second form has no direct parallel in ordinary algebra but is crucial in Boolean algebra).
- **Explanation:** The first law is similar to multiplication distributing over addition. The second law shows that the OR operation can also be distributed over the AND operation.
- **Example (First Law):** Let $A=1$, $B=0$, $C=1$.
 - LHS: $1 \cdot (0 + 1) = 1 \cdot 1 = 1$

- RHS: $(1 \cdot 0) + (1 \cdot 1) = 0 + 1 = 1$

4. Identity Law

This law defines how a variable interacts with the identity elements, 0 and 1.

- **Definition:**
 - $A + 0 = A$
 - $A \cdot 1 = A$
- **Explanation:** ORing a variable with 0 does not change its value. ANDing a variable with 1 also does not change its value.

5. Complement Law (or Inversion Law)

This law defines the relationship between a variable and its inverse.

- **Definition:**
 - $A + A' = 1$
 - $A \cdot A' = 0$
- **Explanation:** A variable ORed with its own complement will always result in 1 (since if A is 0, A' is 1, and vice versa). A variable ANDed with its complement will always result in 0.

6. Idempotent Law

This law states that applying an operation to a variable with itself yields the original variable.

- **Definition:**
 - $A + A = A$
 - $A \cdot A = A$
- **Explanation:** Unlike in regular algebra, adding a variable to itself does not double it. ORing a variable with itself or ANDing it with itself results in the original value.

7. Absorption Law

This law is extremely useful for simplifying complex expressions.

- **Definition:**
 - $A + (A \cdot B) = A$
 - $A \cdot (A + B) = A$
- **Explanation:** In the first form, the term A "absorbs" the term $A \cdot B$. If A is 1, the expression is $1 + (1 \cdot B) = 1$. If A is 0, the expression is $0 + (0 \cdot B) = 0$. In both cases, the result is A. A similar logic applies to the second form.

1.2 DeMorgan's Rules

DeMorgan's rules are two transformative theorems that provide a method for complementing a Boolean expression. They are essential for converting expressions from SOP to POS form and vice-versa and for simplifying circuits, particularly those involving NAND and NOR gates.

DeMorgan's First Law

- **Statement:** The complement of a product of variables is equal to the sum of their individual complements.
- **Formula:** $(A \cdot B)' = A' + B'$
- **Explanation:** This means that a 2-input NAND gate $((A \cdot B)')$ is logically equivalent to an OR gate whose inputs are first inverted (A' and B'). This is often called a "bubbled OR" gate.
- **Proof by Truth Table:**

A	B	A'	B'	$A \cdot B$	$(A \cdot B)'$	$A' + B'$
0	0	1	1	0	1	1
0	1	1	0	0	1	1
1	0	0	1	0	1	1
1	1	0	0	1	0	0

Since the columns for $(A \cdot B)'$ and $A' + B'$ are identical for all possible inputs, the theorem is proven true.

DeMorgan's Second Law

- **Statement:** The complement of a sum of variables is equal to the product of their individual complements.
- **Formula:** $(A + B)' = A' \cdot B'$
- **Explanation:** This means that a 2-input NOR gate $((A + B)')$ is logically equivalent to an AND gate whose inputs are first inverted (A' and B'). This is often called a "bubbled AND" gate.
- **Proof by Truth Table:**

A	B	A'	B'	$A + B$	$(A + B)'$	$A' \cdot B'$
0	0	1	1	0	1	1
0	1	1	0	1	0	0
1	0	0	1	1	0	0

1 1 0 0 1 0 0

The identical output columns prove the theorem.

1.3 Signed Binary Number Representation

Computers must be able to handle both positive and negative numbers. **Signed binary numbers** use the first bit, the **Most Significant Bit (MSB)**, to indicate the sign.

- **MSB = 0:** The number is positive.
- **MSB = 1:** The number is negative.

Let's explore the three main methods using an 8-bit system for all examples.

1. Signed Magnitude Representation

- **Definition:** This is the most straightforward method. The MSB is the sign bit, and the remaining 7 bits represent the magnitude (the absolute value) of the number.
- **Example: Represent +29 and -29**
 1. **Magnitude:** First, find the binary representation of the magnitude, 29.
 $29_{10} = 16 + 8 + 4 + 1 = 11101$ in binary.
 2. **Padding:** In an 8-bit system, we pad the 7 magnitude bits with leading zeros: 0011101.
 3. **+29:** Set the sign bit (MSB) to 0 for positive: 00011101.
 4. **-29:** Set the sign bit (MSB) to 1 for negative: 10011101.
- **Drawbacks:**
 - **Dual Zero:** There are two representations for zero: 00000000 (+0) and 10000000 (-0).
 - **Complex Arithmetic:** Addition and subtraction require separate logic circuits, making the hardware more complex.

2. 1's Complement Representation

- **Definition:** Positive numbers are represented in their standard binary form (with MSB=0). To find the representation of a negative number, you take its positive binary form and invert every bit (change all 1s to 0s and all 0s to 1s).
- **Example: Represent -29**
 1. **Positive Representation:** Start with the 8-bit binary for +29: 00011101.
 2. **Invert all bits:** Flip each bit to get the 1's complement.
 11100010 .
 3. So, -29 in 1's complement is 11100010.
- **Drawbacks:**

- **Dual Zero:** This method also has two representations for zero: 00000000 (+0) and 11111111 (-0).
- **End-Around Carry:** During addition, if a carry is generated from the MSB, it must be added back to the least significant bit, which complicates the arithmetic logic.

3. 2's Complement Representation

- **Definition:** This is the standard method used in virtually all modern computers. Positive numbers are represented normally. A negative number is found by taking the 1's complement of the positive number and then **adding 1**.
- **Example: Represent -29**
 1. **Positive Representation:** Start with +29: 00011101.
 2. **Find 1's Complement:** Invert all the bits: 11100010.
 3. **Add 1:** Add 1 to the 1's complement result.

11100010

+ 1

11100011

4. So, -29 in 2's complement is 11100011.

- **Advantages:**
 - **Single Zero:** Zero has only one representation: 00000000.
 - **Simple Arithmetic:** Subtraction can be performed by addition using the 2's complement of the subtrahend. This means the same hardware (an adder circuit) can be used for both addition and subtraction, simplifying CPU design significantly.

1.4 & 1.5 Simplification Techniques and K-Maps

Introduction to Simplification

A Boolean function can often be represented by multiple different expressions. The goal of simplification is to find the expression that can be implemented with the fewest logic gates, leading to circuits that are cheaper, consume less power, and operate faster. The two primary methods are algebraic manipulation (using the theorems above) and the graphical Karnaugh map (K-map) method.

Standard Forms of Boolean Expressions

1. Sum-of-Products (SOP) Form

- **Definition:** An SOP expression consists of two or more product terms (terms connected by AND) that are then summed (connected by OR).
- **Minterm:** A product term in a canonical SOP expression that includes every variable of the function, either in its normal or complemented form. For a function with n variables, there are 2^n possible minterms.
- **Example:** For a function $F(A, B, C)$, the expression $F = A'BC + AB'C + ABC$ is in SOP form. Each term ($A'BC$, $AB'C$, ABC) is a minterm.

2. Product-of-Sums (POS) Form

- **Definition:** A POS expression consists of two or more sum terms (terms connected by OR) that are then multiplied (connected by AND).
- **Maxterm:** A sum term in a canonical POS expression that includes every variable of the function, either in its normal or complemented form.
- **Example:** For a function $F(A, B, C)$, the expression $F = (A+B+C) \cdot (A'+B+C')$ is in POS form. Each term ($A+B+C$, $A'+B+C'$) is a maxterm.

Minimization using Karnaugh Map (K-map)

A K-map is a graphical tool that provides a systematic method for simplifying Boolean expressions. It is a grid of cells where each cell corresponds to a minterm. The key feature of a K-map is that adjacent cells (including wrap-around cells) differ by only one variable. This adjacency allows us to easily spot opportunities for simplification.

Procedure for SOP Minimization (Grouping the '1's)

1. **Create the Map:** Draw a K-map with 2^n cells, where n is the number of variables. Label the rows and columns using Gray code (00, 01, 11, 10).
2. **Plot the '1's:** Place a '1' in each cell corresponding to a minterm in the function's SOP form.
3. **Group Adjacent '1's:**
 - Create the largest possible rectangular groups of '1's.
 - The number of '1's in a group must be a power of two (1, 2, 4, 8, 16, ...).
 - Groups can overlap.
 - Account for wrap-around adjacency (top row is adjacent to the bottom row; leftmost column is adjacent to the rightmost column).
4. **Derive Simplified Terms:** For each group, determine which variables remain constant across all cells in that group. Write a product term consisting of these constant variables. If a variable is 0 throughout the group, use its complement (e.g., A'). If it is 1, use its normal form (e.g., A).

5. **Final Expression:** The simplified SOP expression is the sum (OR) of all the product terms derived from the groups.

Example: Minimize $F(A,B,C,D) = \Sigma m(0, 1, 2, 4, 5, 6, 8, 9, 12, 13, 14)$

1. **Create and Plot:**

AB\CD	00	01	11	10
00	1	1	0	1
01	1	1	0	1
11	1	1	0	1
10	1	0	0	0

2. **Group the '1's:**

- **Group 1 (Red):** An 8-cell group formed by the first two columns (cells 0, 1, 4, 5, 8, 9, 12, 13).
 - Across these cells, variables A and B change. Variable C changes.
 - The only variable that remains constant is **D=0**. So the term is **D'**.
- **Group 2 (Blue):** A 4-cell group in the corners (cells 0, 2, 8). This group also includes cell 10, but there's no '1' there. We'll group cells 0, 2, 8, and the wrap-around cell m(10) is not present, but we can group cells 0, 2, 4, 6 and 8, 12. Let's re-examine the plot. The plot is:

AB\CD	00	01	11	10
00	1	1	0	1
01	1	1	0	1
11	1	1	0	0
10	1	0	0	1

1. Plot the 1s :

AB\CD	00	01	11	10
00	1	1	0	1
01	1	1	0	1
11	1	1	0	1
10	1	0	0	0

Correction on plot: m(14) is AB=11, CD=10. m(12) is AB=11, CD=00. m(13) is AB=11, CD=01. Let's re-plot.

AB\CD	00	01	11	10
00	1	1	0	1
01	1	1	0	1
11	1	1	0	1
10	1	0	0	0

This plot is still not right.

m(0) = 0000. m(1)=0001. m(2)=0010. m(4)=0100. m(5)=0101. m(6)=0110.
m(8)=1000. m(9)=1001. m(12)=1100. m(13)=1101. m(14)=1110.

Correct Plot:

AB\CD	00	01	11	10
00	1	1	0	1
01	1	1	0	1
11	1	1	0	1
10	1	1	0	0

Wait, $m(9)$ is 1001. Plot is correct. $m(14)=1110$. Plot is correct.

Let's try the simpler example from the previous response for clarity.

Karnaugh Map for $F(A,B,C,D) = \Sigma m(0, 2, 5, 7, 8, 10, 13, 15)$

		CD			
AB	00	01	11	10	
00	1	0	0	1	
01	0	1	1	0	
11	0	1	1	0	
10	1	0	0	1	

Group 1 (Green): A 2x2 square covering minterms (5, 7, 13, 15).

- In this group, A changes from 0 to 1. B is always 1. C changes from 0 to 1. D is always 1.
- The variables that remain constant are $B=1$ and $D=1$.
- **Term: BD**

• **Group 2 (Blue):** A 2x2 square using wrap-around adjacency covering minterms (0, 2, 8, 10).

- In this group, A changes from 0 to 1. B is always 0. C changes from 0 to 1. D is always 0.
- The variables that remain constant are $B=0$ and $D=0$.
- **Term: $B'D'$**

1. **Final Simplified Expression:** $F = B'D' + BD$